

```

sum += fac*cint[n-1];
cint[0]=2.0*sum;
return Chebyshev(cint,a,b);
}

```

Set the constant of integration.

5.9.1 Clenshaw-Curtis Quadrature

Since a smooth function's Chebyshev coefficients c_i decrease rapidly, generally exponentially, equation (5.9.1) is often quite efficient as the basis for a quadrature scheme. As described above, the `Chebyshev` object can be used to compute the integral $\int_a^x f(x)dx$ when many different values of x in the range $a \leq x \leq b$ are needed. If only the single definite integral $\int_a^b f(x)dx$ is required, then instead use the simpler formula, derived from equation (5.9.1),

$$\int_a^b f(x)dx = (b-a) \left[\frac{1}{2}c_0 - \frac{1}{3}c_2 - \frac{1}{15}c_4 - \cdots - \frac{1}{(2k+1)(2k-1)}c_{2k} - \cdots \right] \quad (5.9.3)$$

where the c_i 's are as returned by `chebft`. The series can be truncated when c_{2k} becomes negligible, and the first neglected term gives an error estimate.

This scheme is known as *Clenshaw-Curtis quadrature* [1]. It is often combined with an adaptive choice of N , the number of Chebyshev coefficients calculated via equation (5.8.7), which is also the number of function evaluations of $f(x)$. If a modest choice of N does not give a sufficiently small c_{2k} in equation (5.9.3), then a larger value is tried. In this adaptive case, it is even better to replace equation (5.8.7) by the so-called "trapezoidal" or Gauss-Lobatto (§4.6) variant,

$$c_j = \frac{2}{N} \sum_{k=0}^{N''} f \left[\cos \left(\frac{\pi k}{N} \right) \right] \cos \left(\frac{\pi j k}{N} \right) \quad j = 0, \dots, N-1 \quad (5.9.4)$$

where (N.B.!) the two primes signify that the first and last terms in the sum are to be multiplied by 1/2. If N is doubled in equation (5.9.4), then half of the new function evaluation points are identical to the old ones, allowing the previous function evaluations to be reused. This feature, plus the analytic weights and abscissas (cosine functions in 5.9.4), often give Clenshaw-Curtis quadrature an edge over high-order adaptive Gaussian quadrature (cf. §4.6.4), which the method otherwise resembles.

If your problem forces you to large values of N , you should be aware that equation (5.9.4) can be evaluated rapidly, and simultaneously for all the values of j , by a fast cosine transform. (See §12.3, especially equation 12.4.11. We already remarked that the nontrapezoidal form (5.8.7) can also be done by fast cosine methods, cf. equation 12.4.16.)

CITED REFERENCES AND FURTHER READING:

- Goodwin, E.T. (ed.) 1961, *Modern Computing Methods*, 2nd ed. (New York: Philosophical Library), pp. 78–79.
- Clenshaw, C.W., and Curtis, A.R. 1960, "A Method for Numerical Integration on an Automatic Computer," *Numerische Mathematik*, vol. 2, pp. 197–205.[1]

5.10 Polynomial Approximation from Chebyshev Coefficients

You may well ask after reading the preceding two sections: Must I store and evaluate my Chebyshev approximation as an array of Chebyshev coefficients for a transformed variable y ?

Can't I convert the c_k 's into actual polynomial coefficients in the original variable x and have an approximation of the following form?

$$f(x) \approx \sum_{k=0}^{m-1} g_k x^k, \quad a \leq x \leq b \quad (5.10.1)$$

Yes, you can do this (and we will give you the algorithm to do it), but we caution you against it: Evaluating equation (5.10.1), where the coefficient g 's reflect an underlying Chebyshev approximation, usually requires more significant figures than evaluation of the Chebyshev sum directly (as by `eval`). This is because the Chebyshev polynomials themselves exhibit a rather delicate cancellation: The leading coefficient of $T_n(x)$, for example, is 2^{n-1} ; other coefficients of $T_n(x)$ are even bigger; yet they all manage to combine into a polynomial that lies between ± 1 . Only when m is no larger than 7 or 8 should you contemplate writing a Chebyshev fit as a direct polynomial, and even in those cases you should be willing to tolerate two or so significant figures less accuracy than the roundoff limit of your machine.

You get the g 's in equation (5.10.1) in two steps. First, use the member function `polycofs` in `Chebyshev` to output a set of polynomial coefficients equivalent to the stored c_k 's (that is, with the range $[a, b]$ scaled to $[-1, 1]$). Second, use the routine `pcshft` to transform the coefficients so as to map the range back to $[a, b]$. The two required routines are listed here:

`chebyshev.h`

`VecDoub Chebyshev::polycofs(Int m)`

Polynomial coefficients from a Chebyshev fit. Given a coefficient array `c[0..n-1]`, this routine returns a coefficient array `d[0..n-1]` such that $\sum_{k=0}^{n-1} d_k y^k = \sum_{k=0}^{n-1} c_k T_k(y) - c_0/2$. The method is Clenshaw's recurrence (5.8.11), but now applied algebraically rather than arithmetically.

```
{
    Int k,j;
    Doub sv;
    VecDoub d(m),dd(m);
    for (j=0;j<m;j++) d[j]=dd[j]=0.0;
    d[0]=c[m-1];
    for (j=m-2;j>0;j--) {
        for (k=m-j;k>0;k--) {
            sv=d[k];
            d[k]=2.0*d[k-1]-dd[k];
            dd[k]=sv;
        }
        sv=d[0];
        d[0] = -dd[0]+c[j];
        dd[0]=sv;
    }
    for (j=m-1;j>0;j--) d[j]=d[j-1]-dd[j];
    d[0] = -dd[0]+0.5*c[0];
    return d;
}
```

`pcshft.h`

`void pcshft(Doub a, Doub b, VecDoub_I0 &d)`

Polynomial coefficient shift. Given a coefficient array `d[0..n-1]`, this routine generates a coefficient array `g[0..n-1]` such that $\sum_{k=0}^{n-1} d_k y^k = \sum_{k=0}^{n-1} g_k x^k$, where x and y are related by (5.8.10), i.e., the interval $-1 < y < 1$ is mapped to the interval $a < x < b$. The array `g` is returned in `d`.

```
{
    Int k,j,n=d.size();
    Doub cnst=2.0/(b-a), fac=cnst;
    for (j=1;j<n;j++) {
        d[j] *= fac;           First we rescale by the factor const...
        fac *= cnst;
    }
    cnst=0.5*(a+b);           ...which is then redefined as the desired shift.
}
```